# Using NGSolve for the Course

Browser access and optional local installation

Numerical Methods for Elliptic Problems, Winter Term 2026 Last checked: November 8, 2026

**No installation is required for the course.** All required exercises can be completed directly at the course JupyterLite site (https://michaelneunteufel.github.io/2026_WS_numerical_methods_elliptic_problems/lab/index.html?path=index.ipynb). The exercises that compile native C++ code to implement a custom finite element space are **voluntary**. They require a local NGSolve installation and a C++ compiler until a hosted compiler environment is announced.

## 1 Choose an installation route

**Browser only (JupyterLite, recommended).** Open the course JupyterLite site linked above and work with the released notebooks immediately. This route covers all required exercises and needs no local installation. JupyterLite cannot compile or load new native C++ extensions.

### Browser storage and backups

JupyterLite automatically stores uploaded and edited files in the current browser profile, normally using persistent browser storage. The files are **not** uploaded to the course server or synchronized between devices. Changing or redeploying the course repository normally does not delete a student's local files. However, a locally modified file takes precedence over a newer released file with the same path, so it may hide an update from the course repository. If a released file looks outdated, first perform a hard refresh. If it is still outdated, download any work in that file, delete only its local copy in JupyterLite's file browser, and reload the site. The current released copy should then reappear without removing other uploaded files.

**Download important work regularly.** Browser files are tied to the current browser, profile, device, website address, and browser storage. They can be lost when site data is cleared, in private-browsing sessions, or when moving to another browser or device. Do not use JupyterLite's "Clear Browser Data" command until every important file has been downloaded: it permanently removes all locally stored files and settings for the site.

**Optional local installation with pip.** Choose this route if you prefer working with local files, want to run larger experiments, or want to attempt the voluntary C++ finite element exercise. The current NGSolve package requires Python 3.10 or newer. For this course, Python 3.12 or 3.13 is recommended.

**Build from source.** Intended for advanced users who want to modify NGSolve itself. It requires a compiler, CMake, Git, development libraries, and substantially more disk space and build time.

## 2 Recommended installation with pip

Install a standard 64-bit CPython distribution. On Windows, select the option to add Python to `PATH`. On macOS, both Apple Silicon and Intel systems are supported by the current universal wheel. On Linux, the official wheels currently target x86-64 systems compatible with `manylinux_2_28`, which requires glibc 2.28 or newer.

### Linux and macOS

Open a terminal in the directory where you keep your course files:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install --upgrade ngsolve jupyterlab ipykernel \
    anywidget webgui_jupyter_widgets
```

On Ubuntu or Debian, install the virtual-environment support first if needed:

```
sudo apt install python3-venv
```

Always use `python -m pip`, rather than an unqualified `pip`, so that packages are installed into the currently selected interpreter.

### Windows

Open PowerShell in the directory where you keep your course files:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install --upgrade ngsolve jupyterlab
python -m pip install --upgrade ipykernel anywidget webgui_jupyter_widgets
```

If PowerShell does not allow activation scripts, use Command Prompt and run:

```
.venv\Scripts\activate.bat
```

### Verify the installation

With the virtual environment active, run:

```
python -c "import ngsolve; print(ngsolve.__version__)"
python -c "import ngsolve; print(ngsolve.__file__)"
python -m jupyter lab
```

The printed module path should be inside the active `.venv`. Then create a notebook and execute:

```
from netgen.geom2d import unit_square
from ngsolve import Mesh
from ngsolve.webgui import Draw

mesh = Mesh(unit_square.GenerateMesh(maxh=0.2))
Draw(mesh)
```

A rotatable mesh should appear below the cell. Start JupyterLab with `python -m jupyter lab` from the activated environment. Otherwise the notebook may use a different Python installation.

# 3 Voluntary: compiling C++ for custom finite elements

These kinds of exercises are voluntary and do not count toward the requirements of the exercise class. It cannot currently be completed in the course's browser-only JupyterLite environment. Use a local installation and compiler if you would like to try it.

The course exercise uses NGSolve's `CompilePythonModule`. A full source build of NGSolve is usually unnecessary, but a native C++ compiler must be available in the same terminal from which JupyterLab is started. Use a C++20 compiler compatible with the installed NGSolve build, including its C++ standard library and compiler flags. Using the same compiler family is the simplest choice. The exact version need not be identical. The `ngscxx` wrapper selects the compiler, so setting `CXX` alone may not change the compiler used for this runtime compilation.

- **Windows:** Install Visual Studio 2022 or Build Tools for Visual Studio with the "Desktop development with C++" workload. Start JupyterLab from an x64 Native Tools command prompt if the compiler is not found in an ordinary shell.
- **macOS:** For this runtime-compilation exercise, Apple's Command Line Tools are normally sufficient. Use a recent version and install them with `xcode-select -install`. The full Xcode application is usually not required for this exercise.
- **Linux:** Install a recent C++ compiler and build tools, for example `sudo apt install build-essential cmake` on Ubuntu.

The following notebook cell identifies the Python environment, the compiler recorded by the NGSolve build, and the runtime compiler wrapper:

```
import sys, shutil, ngsolve
import ngsolve.config as cfg

print(sys.executable)
print(ngsolve.__version__, ngsolve.__file__)
print("Build compiler:", cfg.CMAKE_CXX_COMPILER)
print("Compiler wrapper:",
      shutil.which("ngscxx") or shutil.which("ngscxx.bat"))
```

The following notebook cell is a small compiler test:

```
from ngsolve.fem import CompilePythonModule

module = CompilePythonModule(r'''
m.def("answer", []() { return 42; });
''')
assert module.answer() == 42
```

On macOS, compiler errors are printed by Apple Clang with the source line, column, and diagnostic message before NGSolve reports that compilation failed.

The finite element exercise uses the file-based form:

```
from pathlib import Path
from ngsolve.fem import CompilePythonModule

module = CompilePythonModule(
    Path("mymodule.cpp"), init_function_name="mymodule"
)
```

Download both `CppExtensionTest.ipynb` and `mymodule.cpp` from the course's `exercises/ex01/` folder. Keep them together, activate the course environment, and start JupyterLab from that directory. Run all cells: both assertions should pass and a success message should appear.

# 4 Building Netgen/NGSolve from source

This is the advanced route. Platform dependencies and CMake options change over time. Read the current official installation page (https://ngsolve.org/installation.html#getting-the-source) before starting, and do not mix a source build with another NGSolve installation in the same Python environment.

Install the platform prerequisites below first. Use separate source, build, and installation directories and a fresh Python environment. Install the notebook tools in that environment, but do not pip-install NGSolve there. Configure with the same Python interpreter that you intend to use. The examples use `-DBUILD_OCC=ON` to download and build OpenCascade. Omit this option if suitable OpenCascade development files are installed.

```
mkdir ngsuite
cd ngsuite
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install jupyterlab ipykernel \
    anywidget webgui_jupyter_widgets
git clone --recurse-submodules \
    https://github.com/NGSolve/ngsolve.git src
cmake -S src -B build \
  -DCMAKE_INSTALL_PREFIX="${PWD}/install" \
  -DPython3_EXECUTABLE="$(command -v python)" \
  -DBUILD_OCC=ON
cmake --build build --parallel 4 --target install
```

## Platform prerequisites

**Windows.** Install Visual Studio 2022 with C++, Python, CMake, and Git. Configure and build in an x64 Native Tools Command Prompt:

```
mkdir ngsuite
cd ngsuite
py -3.12 -m venv .venv
call .venv\Scripts\activate.bat
python -m pip install --upgrade pip
python -m pip install jupyterlab ipykernel ^
    anywidget webgui_jupyter_widgets
git clone --recurse-submodules ^
    https://github.com/NGSolve/ngsolve.git src
cmake -S src -B build ^
  -DCMAKE_INSTALL_PREFIX="%CD%/install" ^
  -DPython3_EXECUTABLE="%VIRTUAL_ENV%/Scripts/python.exe" ^
  -DBUILD_OCC=ON
cmake --build build --config Release --target install
set NETGENDIR=%CD%\install\bin
set PATH=%NETGENDIR%;%PATH%
set PYTHONPATH=%CD%\install\Lib\site-packages
```

These environment settings apply to the current prompt. Repeat the activation and the three `set` commands when opening a new prompt in `ngsuite`.

**macOS.** The current official source-build instructions list Xcode, CMake, Git, and Python as prerequisites. Follow that page because the requirements may change. The default compiler is Apple Clang. A user-writable `CMAKE_INSTALL_PREFIX` avoids requiring administrator privileges.

**Linux.** Install a recent compiler, Python development headers, CMake, Git, Tcl/Tk development files, OpenGL/X11 development files, and LAPACK. Exact package names depend on the distribution. The official NGSolve installation page gives an Ubuntu package list. For a headless build, the CMake option `-DUSE_GUI=OFF` can reduce GUI-related requirements.

On Linux and macOS, run the following from `ngsuite` with the source build's environment active. This uses the default installation layout. Custom install-directory options require matching paths.

```
NGS_PY_SITE=$(python -c \
  "import os, sysconfig as s; print(os.path.relpath(
s.get_path('platlib'), s.get_path('data')))")
if [ "$(uname)" = Darwin ]; then
  export NETGENDIR="$PWD/install/Contents/MacOS"
  export PYTHONPATH="$PWD/install/Contents/Resources/$NGS_PY_SITE"
else
  export NETGENDIR="$PWD/install/bin"
  export PYTHONPATH="$PWD/install/$NGS_PY_SITE"
fi
export PATH="$NETGENDIR:$PATH"
```

Repeat these settings in each new terminal after activating the source build's environment. Then repeat the Python, WebGUI, and C++ checks. For a source build, the printed NGSolve module path should be inside `ngsuite/install`.

To update later:

```
git -C src pull
git -C src submodule update --init --recursive
cmake --build build --parallel 4 --target install
```

# 5 Troubleshooting checklist

- **No matching distribution found:** check `python -version`, the CPU architecture, and whether a wheel is available on PyPI. Use Python 3.12 or 3.13 for the course setup.
- **Import error after a successful install:** verify `python -m pip show ngsolve` and remove conflicting NGSolve installations from other Python environments.
- **A fresh environment finds an older NGSolve:** print `ngsolve.__file__` as shown in Section 2. If the path is outside `.venv`, open a clean terminal or, on macOS and Linux, run `unset PYTHONPATH`. Then activate the environment and repeat the pip installation.
- **Notebook imports fail:** start JupyterLab from the activated environment or select the matching kernel.
- **WebGUI is blank:** restart the kernel, reload the browser tab, and run `python -m pip install -upgrade webgui_jupyter_widgets anywidget`.
- **CompilePythonModule cannot call the compiler:** verify the compiler in the same terminal before starting JupyterLab. On Windows, use an x64 Native Tools prompt.

## Current official references

Installation and source builds: https://ngsolve.org/installation.html
NGSolve documentation: https://docu.ngsolve.org/latest/
WebGUI: https://docu.ngsolve.org/latest/i-tutorials/appendix-webgui/webgui.html
C++ finite element tutorial: https://docu.ngsolve.org/latest/i-tutorials/unit-9.1-C++FE/CppExtension.html
Binary packages and supported Python versions: https://pypi.org/project/ngsolve/